推荐算法（CTR预测）提分策略汇总

> 本文档整理了推荐系统和CTR预测任务中常见的提分策略，重点关注**不改动模型架构**的优化方法。

一、数据层面优化

1.1 特征工程优化

交叉特征构造

- **二阶交叉特征**：user_id × item_id, category × brand 等
- **高阶交叉特征**：user_age × item_category × time_slot
- **统计特征**：用户历史点击率、物品曝光点击比、转化率等
- **时序特征**：用户最近N次行为的统计（点击数、停留时长、转化次数）

序列特征增强

- **行为序列长度优化**：根据数据分布调整 `seq_max_lens`
- **序列截断策略**：保留最近K个行为 vs 采样关键行为
- **序列特征编码**：位置编码、时间间隔编码

特征选择与过滤

- **特征重要性分析**：使用SHAP值或特征重要性排序
- **低频特征处理**：调整 `emb_skip_threshold` 阈值
- **特征分组**：按业务逻辑分组（用户特征组、物品特征组、上下文特征组）

1.2 数据增强

负采样策略

- **Hard Negative Mining**：选择难分样本作为负样本
- **负样本比例调整**：1:1, 1:2, 1:4 等不同比例实验
- **曝光未点击样本**：使用真实曝光但未点击的样本

数据平衡

- **类别平衡**：对不同类别的样本进行重采样
- **时间窗口平衡**：确保训练集覆盖不同时间段
- **领域平衡**：多领域数据的均衡采样

1.3 数据预处理

归一化策略

- **数值特征标准化**：Z-score, Min-Max, Log变换
- **特征分桶**：连续特征离散化（年龄分段、价格区间）

- **异常值处理**：截断、平滑处理

二、训练策略优化

2.1 优化器调参

学习率策略

- **学习率范围**：1e-5 到 1e-2 之间搜索
- **Warmup策略**：前N个epoch使用较小学习率预热
- **学习率衰减**：StepLR, CosineAnnealing, ReduceLROnPlateau
- **分层学习率**：Embedding层使用较小学习率，上层使用较大学习率

优化器选择

- **AdamW**：默认推荐，weight_decay=1e-4 到 1e-2
- **Muon**：新型优化器，适合大规模训练
- **Adagrad**：适合稀疏特征（Embedding层）
- **混合优化器**：Dense参数用AdamW，Sparse参数用Adagrad

2.2 正则化技术

Dropout

- **Dropout率调整**：0.1 到 0.5 之间实验
- **分层Dropout**：不同层使用不同Dropout率
- **DropConnect**：随机丢弃连接而非神经元

Weight Decay

- **L2正则化**：weight_decay 从 1e-5 到 1e-2
- **自适应正则化**：根据特征频率调整正则化强度

Label Smoothing

- **平滑系数**：0.05 到 0.2 之间
- **减少过拟合**：防止模型对训练集过度自信

2.3 损失函数优化

损失函数选择

- **Binary Cross Entropy**：标准CTR预测损失
- **Focal Loss**：处理样本不平衡问题
- **Dice Loss**：适合极度不平衡数据
- **InfoNCE Loss**：对比学习损失，增强表征能力

多任务学习

- **辅助任务**：同时预测点击、转化、停留时长
- **任务权重平衡**：动态调整不同任务的损失权重

2.4 训练技巧

Batch Size调整

- **大Batch**：256, 512, 1024（需配合学习率调整）
- **梯度累积**：小显存时模拟大Batch效果

混合精度训练（AMP）

- **FP16训练**：加速训练，减少显存占用
- **动态Loss Scaling**：防止梯度下溢

Early Stopping

- **Patience调整**：3到10个epoch
- **监控指标**：AUC优先，LogLoss辅助

Embedding初始化

- **预训练Embedding**：使用Word2Vec、Item2Vec预训练
- **初始化策略**：Xavier, He初始化
- **周期性重初始化**：`reinit_sparse_after_epoch` 策略

三、验证与评估优化

3.1 验证策略

数据划分

- **时间切分**：按时间顺序划分训练/验证集（更符合线上场景）
- **验证集比例**：10%到20%
- **交叉验证**：K-fold验证（计算成本高）

验证频率

- **Epoch级验证**：每个epoch结束验证
- **Step级验证**：`eval_every_n_steps` 设置为100-500
- **早期密集验证**：前几个epoch频繁验证，后期降低频率

3.2 评估指标

主要指标

- **AUC**：排序能力评估
- **LogLoss**：概率校准评估
- **GAUC**：分组AUC，更稳定

辅助指标

- **Precision@K / Recall@K**：Top-K推荐效果
- **NDCG**：排序质量
- **Coverage**：推荐覆盖率

四、模型集成策略

4.1 单模型集成

Checkpoint Ensemble

- **多个Checkpoint平均**：选择验证集上最好的3-5个checkpoint
- **指数移动平均（EMA）**：训练过程中维护参数的EMA

Test Time Augmentation（TTA）

- **特征扰动**：对输入特征添加小噪声后多次预测
- **Dropout TTA**：测试时保持Dropout，多次预测取平均

4.2 多模型集成

不同超参数模型

- **学习率不同**：训练3-5个不同学习率的模型
- **随机种子不同**：不同初始化的模型

集成方法

- **简单平均**：多个模型预测结果平均
- **加权平均**：根据验证集AUC加权
- **Stacking**：用LR或GBDT对多模型输出进行二次建模

五、实验管理与调参

5.1 超参数搜索

搜索策略

- **Grid Search**：网格搜索（计算成本高）
- **Random Search**：随机搜索（效率更高）
- **Bayesian Optimization**：贝叶斯优化（智能搜索）

关键超参数优先级

1. **学习率**（最重要）
2. **Batch Size**
3. **Weight Decay**

4. **Dropout Rate**
5. **Embedding维度**

5.2 实验记录

必须记录的信息

- **超参数配置**：完整的训练参数
- **数据版本**：训练数据的版本或时间范围
- **评估指标**：训练集、验证集、测试集的所有指标
- **训练时间**：每个epoch的时间、总训练时间

工具推荐

- **TensorBoard**：可视化训练曲线
- **Weights & Biases**：实验管理平台
- **MLflow**：实验跟踪

六、针对你的代码的具体建议

基于你的 `baseline` 代码，以下是可以直接尝试的提分策略：

6.1 立即可尝试（修改run.sh参数）

```
# 1. 调整学习率
--learning_rate 0.001  # 默认可能是0.0001，可以尝试0.0005, 0.001, 0.002

# 2. 调整Batch Size
--batch_size 512  # 如果显存允许，尝试更大的batch

# 3. 增加验证频率
--eval_every_n_steps 200  # 更频繁地监控训练效果

# 4. 调整Weight Decay
--weight_decay 0.01  # 尝试0.001, 0.01, 0.1

# 5. 调整序列长度
--seq_max_lens seq_a:128,seq_b:128,seq_c:256,seq_d:256  # 根据数据分布调整

# 6. 调整Embedding维度
--d_model 128  # 尝试64, 128, 256

# 7. 调整稀疏特征阈值
--emb_skip_threshold 500000  # 降低阈值包含更多特征
```

6.2 需要小改动代码

1. **启用Label Smoothing**：如果代码支持，添加 `--label_smoothing 0.1`

2. **切换损失函数**：尝试Focal Loss或Dice Loss
3. **启用混合精度训练**：添加AMP支持
4. **调整优化器**：尝试不同的优化器组合

6.3 实验流程建议

第一轮：学习率搜索

```
for lr in 0.0005 0.001 0.002 0.005; do
    bash run.sh --learning_rate $lr --num_epochs 5
done
```

第二轮：基于最佳学习率调整其他参数

```
# 假设最佳lr=0.001
bash run.sh --learning_rate 0.001 --batch_size 512 --num_epochs 10
bash run.sh --learning_rate 0.001 --weight_decay 0.01 --num_epochs 10
bash run.sh --learning_rate 0.001 --d_model 128 --num_epochs 10
```

第三轮：组合最佳参数

```
bash run.sh --learning_rate 0.001 --batch_size 512 --weight_decay 0.01 --
num_epochs 20
```

七、常见问题与解决方案

7.1 过拟合问题

症状：训练集AUC高，验证集AUC低

解决方案：

- 增大Weight Decay
- 增大Dropout
- 减少模型复杂度（降低d_model）
- 增加训练数据
- 使用Label Smoothing

7.2 欠拟合问题

症状：训练集和验证集AUC都低

解决方案：

- 增大模型容量（增加d_model）
- 降低正则化强度
- 增加训练轮数
- 提高学习率
- 增加特征交叉

7.3 训练不稳定

症状：Loss震荡，AUC波动大

解决方案：

- 降低学习率
- 使用学习率Warmup
- 增大Batch Size
- 检查数据是否有异常值
- 使用梯度裁剪

7.4 收敛过慢

症状：训练很多epoch仍在缓慢提升

解决方案：

- 提高学习率
- 使用更好的优化器（AdamW）
- 检查特征是否需要归一化
- 使用预训练Embedding

八、参考资源

学术论文

- Enhancing CTR Prediction through Sequential Recommendation Pre-training
- Enhancing Click-Through Rate Prediction Accuracy with Cross-Stage Data
- Exploring Scaling Laws of CTR Model for Online Performance Improvement
- Optimizing CTR Prediction Models with Frequency-wise Hessian Eigenvalue Regularization
- Optimizing Feature Set for Click-Through Rate Prediction

技术博客

- 深度学习用于推荐系统
- 推荐系统遇上深度学习(十九)--探秘阿里之深度兴趣网络(DIN)
- 深度学习调参手册

工业实践

- Feature Staleness Aware Incremental Learning for CTR Prediction
- A Universal Framework for Compressing Embeddings in CTR Prediction
- Rethinking Transformers for CTR Prediction

九、提分优先级总结

🔥 高优先级（立即尝试）

1. **学习率调优**：最重要的超参数
2. **Batch Size调整**：配合学习率一起调
3. **验证策略优化**：确保验证集能反映真实效果
4. **Early Stopping调整**：避免过拟合

⭐ 中优先级（有一定收益）

5. **Weight Decay调整**：控制过拟合
6. **损失函数优化**：尝试Focal Loss、Dice Loss
7. **特征工程**：增加交叉特征、统计特征
8. **序列长度优化**：根据数据分布调整

💡 低优先级（锦上添花）

9. **模型集成**：多模型平均
10. **TTA**：测试时增强
11. **Embedding初始化**：预训练Embedding
12. **混合精度训练**：加速训练

十、实验Checklist

在开始实验前，确保：

- ☐ 记录当前baseline的AUC和LogLoss
- ☐ 准备好实验记录表格（超参数、指标、备注）
- ☐ 确保验证集划分合理（时间切分）
- ☐ 设置合理的Early Stopping（避免浪费时间）
- ☐ 每次只改变一个变量（控制变量法）
- ☐ 保存每个实验的checkpoint
- ☐ 记录训练时间和资源消耗

最后建议：从简单到复杂，先把学习率、Batch Size、Weight Decay这三个最重要的超参数调好，通常就能获得明显提升。不要一次改太多参数，否则无法判断哪个改动有效。

祝实验顺利！🚀